MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)

Department of Computer Applications

Mini Project Report

REAL-TIME NETWORK INTRUSION DETECTION AND ALERT SYSTEM

Done by

ABHINAV SAJEEV

Reg No: MAC25MCA-2002

Under the guidance of

Dr. Sonia Abraham

2025–2027

MAR ATHANASIUS COLLEGE OF ENGINEERING

KOTHAMANGALAM

(Government Aided Autonomous Institution)

CERTIFICATE

Real-Time Network Intrusion Detection and Alert System

Certified that this is the bonafide record of project work done by

ABHINAV SAJEEV

Reg No: MAC25MCA-2002

during the third semester, in partial fulfilment of requirements for award of the degree,

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Dr. Sonia Abraham
Faculty Guide

Dr. Sonia Abraham
Head of the Department

Dr. Manu John
Project Coordinator

Internal Examiners

Acknowledgement

First and foremost, I express my gratitude to God Almighty for showering blessings and granting the strength, perseverance, and wisdom to complete this mini-project work successfully.

I express my profound gratitude and heartfelt thanks to Dr. Sonia Abraham, Head of the Department of Computer Applications, Mar Athanasius College of Engineering, Kothamangalam, for providing excellent academic facilities, lab resources, and continuous encouragement throughout this endeavour.

I extend my deep gratitude and sincere appreciation to my project guide, Dr. Sonia Abraham, for invaluable guidance, constructive suggestions, insightful feedback, and patient support rendered during all stages of this project development.

I express my sincere gratitude to the Project Coordinators, Dr. Manu John and Prof. Minu Maria Joseph, as well as the faculty members and technical staff of the Department of Computer Applications, for their kind cooperation, valuable guidance, and assistance.

Finally, I express my deepest appreciation to my parents, family members, and friends for their continuous moral support, understanding, and encouragement throughout the completion of this project.

Abhinav Sajeev

Abstract

Network security has become increasingly critical as modern cyberattacks grow in frequency, stealth, and complexity, threatening organizational digital infrastructure across all sectors. Traditional signature-based intrusion detection systems struggle to detect novel zero-day exploits, evasive low-rate probing, and evolving volumetric attack vectors. Machine learning offers a promising paradigm for adaptive anomaly detection, yet operational implementations frequently encounter significant challenges, including severe class imbalance between benign and malicious flows, prohibitive inference latency, and the absence of end-to-end integration between theoretical offline models and live packet capture pipelines.

This work presents the Real-Time Network Intrusion Detection and Alert System (NIDAS), an end-to-end machine learning platform engineered for continuous network monitoring and automated threat alerting. The system is trained and evaluated on a curated corpus of 1,799,905 network flows derived from the CIC-IDS2017 benchmark, covering Normal Traffic alongside four prevalent attack vectors: Denial of Service (DoS), Distributed Denial of Service (DDoS), Port Scanning, and Brute Force. A rigorous five-stage feature selection methodology, comprising zero-variance removal, collinearity reduction ($r > 0.90$), Kruskal-Wallis non-parametric significance testing, Random Forest feature importance scoring, and domain validation, prunes 77 raw flow features down to an optimal 40-feature operational space.

To overcome the operational limitations of single-model classifiers, NIDAS employs a soft-voting ensemble coupling Random Forest and Extreme Gradient Boosting (XGBoost). Random Forest ($B = 200$ trees) provides variance dampening across noisy packet metrics, while XGBoost (300 estimators with histogram-based binning) supplies precise bias reduction for difficult boundary distinctions. Balanced sample weighting counters class imbalance without introducing synthetic oversampling artifacts. On an independent test set of 394,168 flows, the ensemble achieves 99.88% overall accuracy, a macro-averaged F1-score of 99.76%, and sub-millisecond per-flow inference latency (< 0.6 ms).

The operational pipeline is deployed as a lightweight client-server application featuring asynchronous live packet capture via Npcap/libpcap, online feature extraction using Welford's algorithm, and a stateful cross-flow sliding window detector for stealthy port scans. Inferred threats are categorized into color-coded severity tiers (Critical, High, Medium) and streamed via FastAPI WebSockets to an interactive React SOC dashboard with automated self-traffic suppression. The result is a robust, production-ready solution that bridges theoretical intrusion detection with low-overhead, real-time security operations.

List of Tables

2.1 Summary of Paper 1 3
2.2 Summary of Paper 2 5
2.3 Summary of Paper 3 7
2.4 Literature Summary 8

3.1 Flow Volume and Packet Length Statistics (Features 1–10) 15
3.2 Flow Throughput and Inter-Arrival Timing Dynamics (Features 11–20) 16
3.3 Directional Timing and Protocol Header Attributes (Features 21–30) 17
3.4 TCP Control Flags, Windowing, and State Dynamics (Features 31–40) 18
3.5 Software Environment 37
3.6 Hardware Environment 38

4.1 Empirical Model Evolution and Validation Performance across Milestones 45

5.1 Model Comparison on Test Set (394,168 Flows) 47
5.2 Per-Class Evaluation Metrics - Ensemble (Test Set) 49
5.3 TP, FP, FN, TN, FPR and FNR per Class - Ensemble (Test Set) 50

10.1 Minimum Software Requirements 63
10.2 Minimum Hardware Requirements 64

List of Figures

3.1 Feature Distribution Across Attack Families (Log1p Scale) 19
3.2 Class Distribution 20
3.3 Feature Correlation Matrix (sample n=15,000) 21
3.4 Packet Length Mean Density Distribution Across Attack Classes 22
3.5 Flow Duration Density Distribution Across Attack Classes 23
3.6 Flow Bytes/s Density Distribution Across Attack Classes 24
3.7 Forward Packet Length Mean Density Distribution Across Attack Classes 25
3.8 Packet Length Mean Distribution by Attack Class 26
3.9 Flow Duration Distribution by Attack Class (Log Scale) 27
3.10 Flowchart of Random Forest 29
3.11 Flowchart of XGBoost 31
3.12 Project Pipeline 34

4.1 Feature Selection and Preprocessing 39
4.2 Scaling and Label Encoding 40
4.3 Data Partitioning and Class-Weight Adjustment Code 41
4.4 Base Classifier Implementation Code (Random Forest and XGBoost) 42
4.5 Ensemble Code 43
4.6 Top 20 Random Forest Feature Importances 44
4.7 Model Evaluation Code (Classification Report and Confusion Matrix) 46

5.1 Confusion Matrix - Ensemble (Test Set) 48
5.2 Per-Class Classification Metrics Heatmap - Ensemble 50

6.1 Detailed Flow Inspection and Class Probability Distribution 55
6.2 Real-Time Network Intrusion Monitoring Dashboard 56

7.1 Git Repository 57

Contents

Acknowledgement i

Abstract ii

List of Tables iii

List of Figures iv

1 Introduction 1

2 Supporting Literature 2

2.1 Literature Review 2

2.2 Literature Review Summary 8

2.3 Findings and Proposals 9

3 System Analysis 10

3.1 Analysis of the Dataset 10

3.1.1 About the Dataset 10

3.1.2 Explore the Dataset 11

3.2 Data Preprocessing 12

3.2.1 Data Cleaning 12

3.2.2 Analysis of Feature Variables 12

3.2.3 Analysis of Class Variables 18

3.3 Data Visualization 20

3.4 Analysis of Architecture and Algorithms 27

3.4.1 Detailed Study Report of the Proposed Algorithms 28

3.5 Project Pipeline 33

3.5.1 Project Pipeline Diagram and Development Plan 33

3.6 Feasibility Analysis 36

3.6.1 Technical Feasibility 36

3.6.2 Economic Feasibility 36

3.6.3 Operational Feasibility 37

3.7 System Environment 37

3.7.1 Software Environment 37

3.7.2 Hardware Environment 38

4 System Design 39

4.1 Model Building 39
4.1.1 Implementation Code 39
4.1.2 Model Planning 40
4.1.3 Training 41
4.1.4 Model Evolution and Validation Across Development Milestones 44
4.1.5 Testing 45
5 Results and Discussion 47
5.1 Comparison of Models 47
5.2 Confusion Matrix 48
5.3 Evaluation Metrics 49
5.4 Metrics Calculated from the Confusion Matrix 50
5.5 Discussion 51
6 Model Deployment 52
6.1 Model Implementation 52
6.2 UI Design 54
7 Git History 57
8 Conclusions 58
8.1 Implementation of the Real-Time IDS Pipeline 58
8.2 Pipeline Integrity and Training Inference Parity 58
8.3 Domain Generalization and Continuous Adaptation 59
8.4 Critical Analysis of Operational Limitations 59
8.5 Concluding Remarks 60
9 Future Work 61
9.1 Continual Learning and Adaptive Concept Drift Mitigation 61
9.2 Explainable AI (XAI) and Real-Time SHAP Attributions 61
9.3 Hardware Acceleration and Kernel-Level Packet Inspection 61
9.4 Hybrid Unsupervised Zero-Day Threat Interception 62
10 Appendix 63
10.1 Minimum Software Requirements 63
10.2 Minimum Hardware Requirements 64
11 References 65

1. Introduction

The rapid expansion of digital infrastructure has made computer networks a primary target for malicious actors. Global cybercrime damages are projected to exceed ten trillion dollars annually in 2025–2026, with India seeing sharp rises in DDoS and ransomware attacks targeting banking, healthcare, and government networks.

An Intrusion Detection System (IDS) monitors network traffic for unauthorized access or malicious behaviour. Signature-based systems match traffic against known attack patterns; anomaly-based systems flag deviations from a learned baseline. Signature-based approaches fail against zero-day exploits, driving research into ML-based detection capable of generalizing to unseen attacks.

A persistent gap exists between academic IDS research and practical deployment. Most published systems evaluate on static datasets without addressing real-time operational pipelines, continuous alerting, or analyst-facing interfaces, limiting operational value. A further challenge is severe class imbalance: benign flows dominate real traffic, and models that ignore this risk high false-negative rates on minority attack classes.

This project is motivated by CERT-In mandates for faster incident detection and the expanded network perimeter from hybrid cloud and remote work environments.

Objectives:

1. Train a Random Forest + XGBoost ensemble on the combined CIC-IDS2017 and CSE-CIC-IDS2018 benchmark for multi-class intrusion detection.
2. Implement a real-time alert mechanism upon detection of anomalous flows.
3. Build a real-time detection pipeline: live flow feature extraction → ensemble classification → alerting and telemetry.
4. Comparatively evaluate RF, XGBoost, and the ensemble on accuracy, precision, recall, and F1.

NIDAS is designed as an end-to-end real-time intrusion detection pipeline that processes network traffic via a custom CICFlowMeter implementation. Each flow is classified into: Normal Traffic, DoS, DDoS, Port Scanning, or Brute Force. Detected anomalies trigger timestamped alerts with predicted attack type and confidence score. A React JS dashboard surfaces live flow classifications; a FastAPI backend handles communication between the detection engine and UI.

The key contribution is end-to-end integration of ensemble detection with real-time alerting and dashboard visualization operationally actionable, not just accurate.